Sistema de Gestión de Citas y Control de Pacientes para Centros de Salud de Cartago

Avance 2 del proyecto – Segundo corte

Saray Foronda Restrepo

Corporación de Estudios Tecnológicos del Norte del Valle
Ingeniería de Sistemas
Bases de Datos II

Docente: Jhon James Cano Sánchez

Cartago, Valle del Cauca
25 de octubre de 2026

Video de sustentación:
https://drive.google.com/file/d/1HNi2bk2MWJ6LJHqQXN-8wpyttmbwKF1h/view?usp=drive_link

Repositorio (etiqueta avance-2):
https://github.com/forondasaray4-droid/proyecto-BDII

1. Descripción del Proyecto

En Cartago (Valle del Cauca) el agendamiento de citas médicas todavía depende en gran medida de la atención presencial: muchas personas pierden un día completo haciendo fila y quienes viven fuera del casco urbano deben desplazarse solo para realizar el trámite. El proyecto resuelve esta necesidad con una base de datos relacional en MariaDB, ejecutada en Docker, que centraliza pacientes, médicos, citas, historial médico y recordatorios automáticos de los centros de salud del municipio.

Usuarios del sistema. El personal de recepción agenda y cancela citas; los médicos consultan su agenda y registran el historial; los pacientes consultan sus datos con un PIN; y el administrador gestiona la información y la seguridad. **Alcance:** en este corte se construyó la lógica en el servidor (procedimientos, funciones, triggers), las vistas y la seguridad por roles. No incluye aún una interfaz completa ni el envío real de mensajes: los recordatorios quedan registrados en estado Pendiente.

Cambios desde el primer avance:

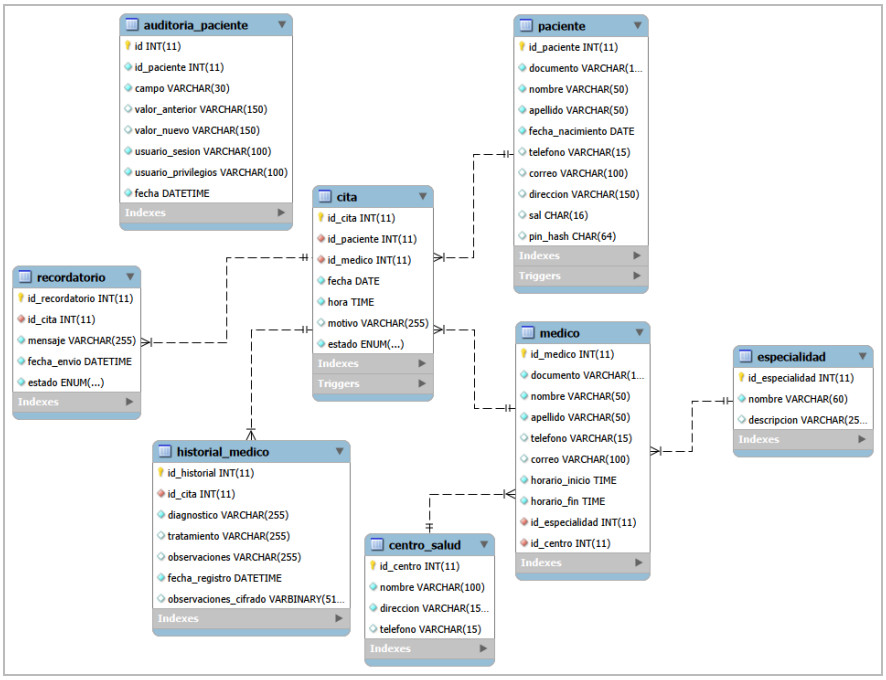
- Los procedimientos `sp_registrar_paciente`, `sp_agendar_cita` y `sp_cancelar_cita` ahora validan con `SIGNAL`; `sp_agendar_cita` usa `LAST_INSERT_ID()` en lugar de `MAX()`, que podía devolver el id de otra sesión.
- Se agregaron cinco funciones almacenadas, dos vistas, un trigger de auditoría y la tabla `auditoria_paciente`.
- Se implementó seguridad: tres roles, tres usuarios, hash con sal del PIN, cifrado AES de las observaciones médicas y política de contraseñas.
- Se añadió `cargar_bd.sh` para crear toda la base desde cero con un solo comando.
- Retroalimentación del docente en el primer avance: convertir los estados de la cita en una tabla independiente y agregar franjas horarias por médico. Ambas mejoras se planean para el proyecto final (sección 8).

2. Modelo de Datos

La Figura 1 muestra el modelo entidad-relación actualizado, obtenido por ingeniería inversa en MySQL Workbench sobre la base `proyecto_citas`. Respecto al primer avance se agregó la tabla `auditoria_paciente` y las columnas `sal`, `pin_hash` (en `paciente`) y `observaciones_cifrado` (en `historial_medico`). La tabla `auditoria_paciente` no tiene llave foránea a propósito: es un registro histórico y debe conservarse aunque el paciente cambie o se elimine. La relación entre `cita` e `historial_medico` es 1:1 porque `id_cita` tiene restricción UNIQUE, aunque Workbench la dibuja como 1:N.

Figura 1

Modelo entidad-relación de la base de datos proyecto_citas



Nota. Elaboración propia en MySQL Workbench.

Tabla 1

Tablas del modelo y su propósito

Tabla	Propósito
`centro_salud`	Centros de salud del municipio donde se prestan las atenciones.
`especialidad`	Especialidades médicas ofrecidas.
`medico`	Personal médico, con su especialidad, centro y horario de atención.
`paciente`	Personas que solicitan atención; incluye `sal` y `pin_hash` para el acceso con PIN.
`cita`	Citas entre un paciente y un médico, con fecha, hora y estado.
`historial_medico`	Diagnóstico, tratamiento y observaciones cifradas de cada cita atendida (1:1 con cita).
`recordatorio`	Notificaciones generadas automáticamente un día antes de cada cita.
`auditoria_paciente`	Registro de los cambios sobre datos del paciente: valor anterior y nuevo, usuario y fecha.

3. Procedimientos Almacenados

El proyecto tiene seis procedimientos (Tabla 2). Los tres que se documentan con ejecución exitosa y con error validan sus parámetros con `SIGNAL SQLSTATE '45000'`, de modo que el error llega al cliente con un mensaje propio en lugar de un error genérico de la base de datos. El código completo está en `sql/procedimientos.sql`.

Tabla 2

Procedimientos almacenados del proyecto

Procedimiento	Parámetros	Validaciones con SIGNAL
`sp_registrar_paciente`	IN documento, nombre, apellido, fecha de nacimiento, teléfono, correo, dirección; OUT p_id_paciente	Datos obligatorios vacíos, documento duplicado, fecha de nacimiento futura
`sp_agendar_cita`	IN paciente, médico, fecha, hora, motivo; OUT p_id_cita	Paciente o médico inexistente, fecha pasada
`sp_cancelar_cita`	IN p_id_cita	Cita inexistente o que no está Programada
`sp_reporte_atenciones_centro`	IN p_id_centro; OUT p_total_atendidas	Ninguna (consulta con 5 JOIN)
`sp_acumular_citas_paciente`	IN p_id_paciente; INOUT p_total	Ninguna
`sp_agendar_cita_segura`	IN documento, médico, fecha, hora, motivo	Paciente o médico inexistente, fecha pasada, fuera de horario (SQL SECURITY DEFINER)

3.1 sp_registrar_paciente

Registra un paciente nuevo y devuelve su identificador. Rechaza documentos repetidos y fechas de nacimiento futuras.

```
IF EXISTS (SELECT 1 FROM paciente WHERE documento = p_documento) THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Ya existe un paciente con ese documento';
END IF;
...
INSERT INTO paciente (...) VALUES (...);
SET p_id_paciente = LAST_INSERT_ID();
```

Figura 2

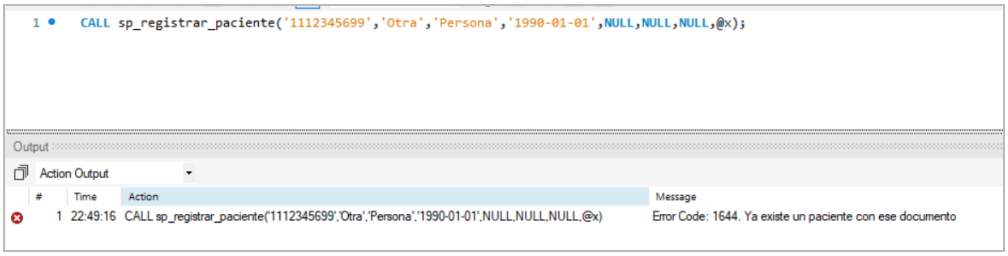
Ejecución exitosa de sp_registrar_paciente (devuelve el id 12)



Nota. Captura propia en MySQL Workbench.

Figura 3

Error de validación en sp_registrar_paciente: documento duplicado



Nota. Captura propia en MySQL Workbench.

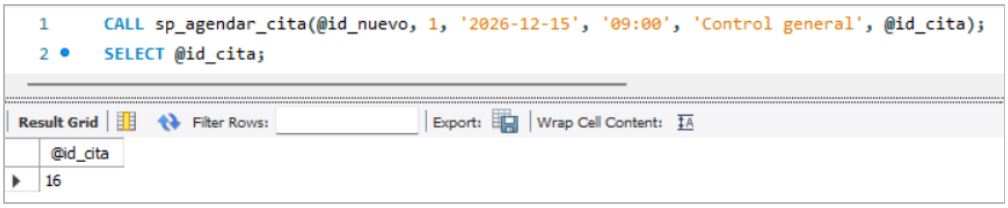
3.2 sp_agendar_cita

Crea la cita en estado Programada y devuelve su id. Valida que el paciente y el médico existan y que la fecha no sea pasada; la disponibilidad y el horario los valida el trigger `trg_validar_cita` (sección 5).

```
IF NOT EXISTS (SELECT 1 FROM paciente WHERE id_paciente = p_id_paciente) THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El paciente no existe';
END IF;
...
INSERT INTO cita (id_paciente, id_medico, fecha, hora, motivo) VALUES (...);
SET p_id_cita = LAST_INSERT_ID();
```

Figura 4

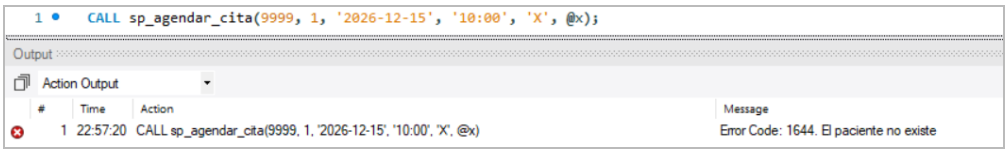
Ejecución exitosa de sp_agendar_cita (devuelve el id 16)



Nota. Captura propia en MySQL Workbench.

Figura 5

Error de validación en sp_agendar_cita: paciente inexistente



Nota. Captura propia en MySQL Workbench.

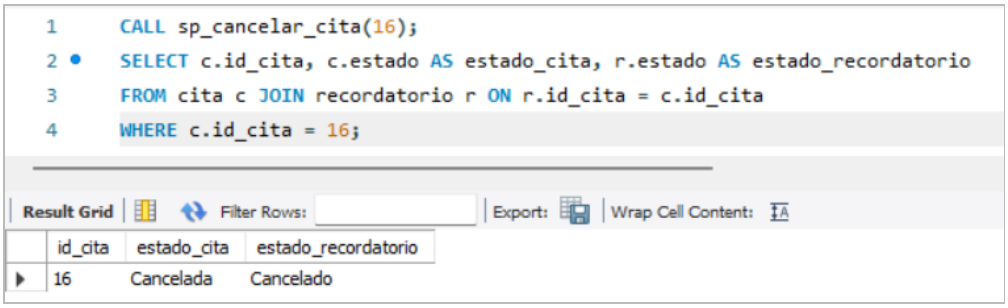
3.3 sp_cancelar_cita

Cancela una cita y, dentro de una transacción, marca como Cancelado su recordatorio pendiente. Solo permite cancelar citas en estado Programada.

```
IF v_estado IS NULL THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La cita no existe';
ELSEIF v_estado <> 'Programada' THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Solo se pueden cancelar citas en estado Programada';
END IF;
```

Figura 6

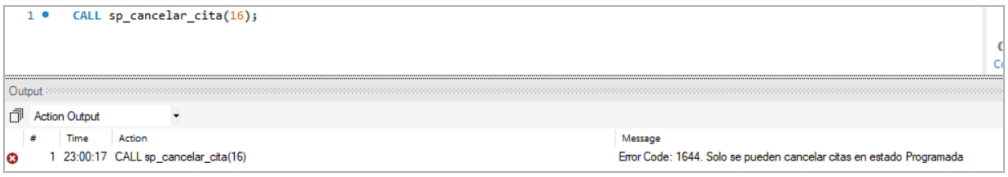
Ejecución exitosa de sp_cancelar_cita: cita y recordatorio cancelados



Nota. Captura propia en MySQL Workbench.

Figura 7

Error de validación en sp_cancelar_cita: la cita ya no está Programada



Nota. Captura propia en MySQL Workbench.

4. Funciones Almacenadas

Se crearon cinco funciones (Tabla 3). Las dos primeras se muestran en uso dentro de un SELECT; el resto se usan en las vistas, en la validación de ingreso y en la agenda (la lista de funciones creadas está en el Anexo C). Todo el código está en `sql/funciones.sql` y `sql/clase10_seguridad_citas.sql`.

Tabla 3

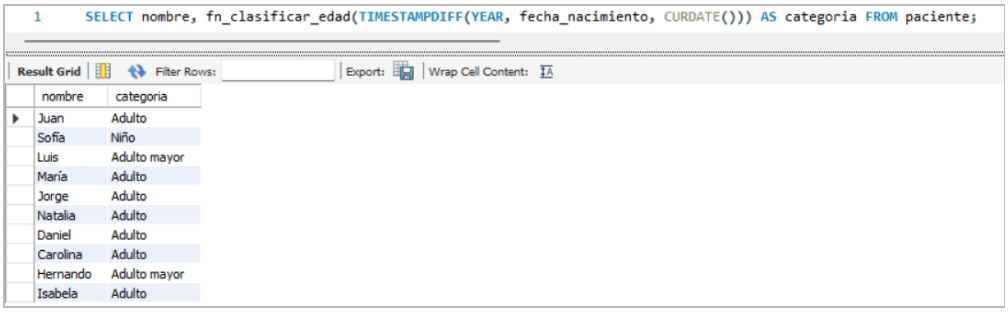
Funciones almacenadas del proyecto

Función	Propósito	Parámetros	Retorno
`fn_clasificar_edad`	Clasifica a una persona por su edad	p_edad INT	VARCHAR: Niño, Adolescente, Adulto, Adulto mayor (NULL si es inválida)
`fn_contar_citas_paciente`	Cuenta las citas de un paciente	p_documento VARCHAR	INT: total de citas (0 si no existe)
`fn_medico_disponible`	Indica si el médico atiende y está libre a esa hora	p_id_medico, p_fecha, p_hora	TINYINT: 1 disponible, 0 no
`fn_enmascarar`	Oculta un dato dejando visibles los últimos caracteres	p_texto, p_visibles	VARCHAR enmascarado
`fn_validar_login_citas`	Valida el PIN del paciente comparando hashes con sal	p_documento, p_pin	TINYINT: 1 válido, 0 inválido

```
CREATE FUNCTION fn_contar_citas_paciente(p_documento VARCHAR(15))
RETURNS INT READS SQL DATA
BEGIN
  DECLARE v_total INT;
  SELECT COUNT(*) INTO v_total FROM cita c
  INNER JOIN paciente p ON c.id_paciente = p.id_paciente
  WHERE p.documento = p_documento;
  RETURN v_total;
END
```

Figura 8

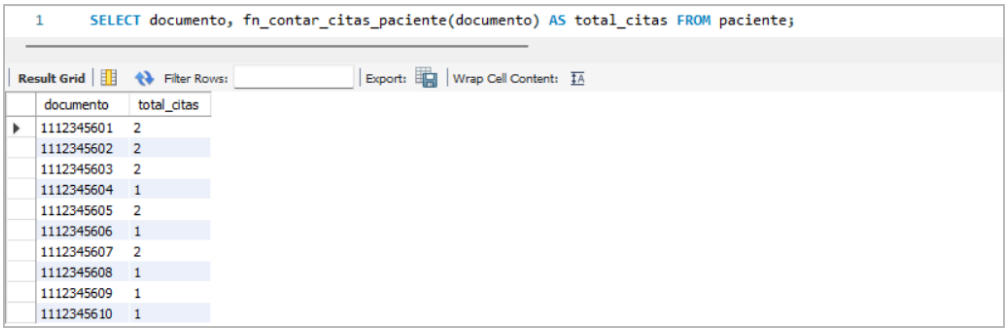
Uso de fn_clasificar_edad dentro de un SELECT



Nota. Captura propia en MySQL Workbench.

Figura 9

Uso de fn_contar_citas_paciente dentro de un SELECT



Nota. Captura propia en MySQL Workbench.

5. Triggers

Tabla 4

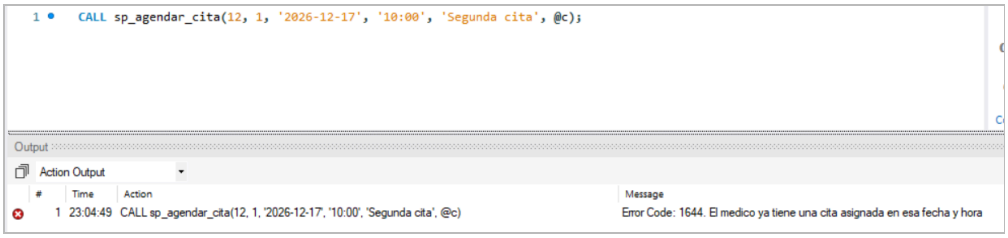
Triggers del proyecto

Trigger	Evento y momento	Función
`trg_validar_cita`	INSERT en `cita`, BEFORE	Rechaza la cita si el médico ya tiene otra activa a esa fecha y hora, o si la hora está fuera de su horario. Es BEFORE porque debe impedir que el registro se guarde.
`trg_generar_recordatorio`	INSERT en `cita`, AFTER	Crea el recordatorio un día antes de la cita. Es AFTER porque necesita el id de la cita ya insertada.
`trg_auditar_paciente_update`	UPDATE en `paciente`, AFTER	Auditoría: registra campo, valor anterior y nuevo, `USER()`, `CURRENT_USER()` y fecha. El PIN nunca se copia a la auditoría.

5.1 trg_validar_cita

Figura 10

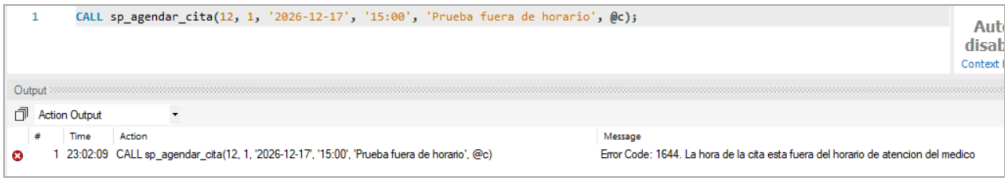
Rechazo de una cita por médico ocupado



Nota. Captura propia en MySQL Workbench.

Figura 11

Rechazo de una cita fuera del horario del médico

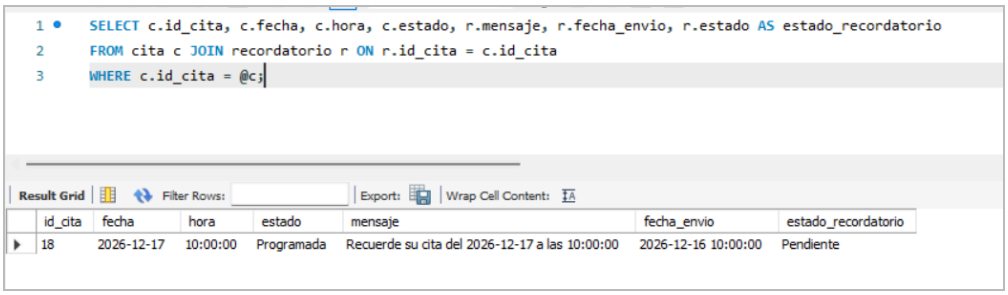


Nota. Captura propia en MySQL Workbench.

5.2 trg_generar_recordatorio

Figura 12

Recordatorio generado automáticamente para la cita 18



Nota. Captura propia en MySQL Workbench.

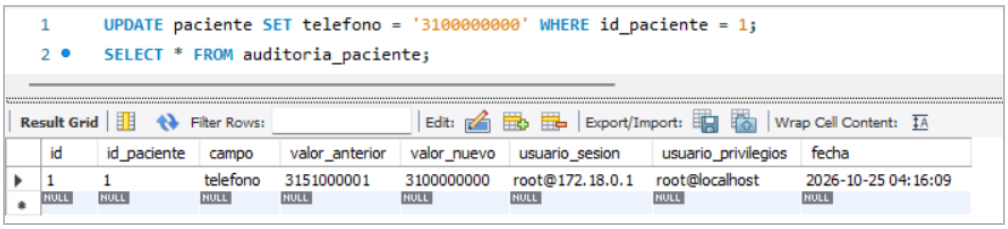
5.3 trg_auditar_paciente_update (auditoría)

Distingue `USER()` (quién inició la sesión, con su host de origen) de `CURRENT_USER()` (la cuenta cuyos privilegios se aplican). En la Figura siguiente, el cambio hecho por root queda con ambos valores.

```
IF NOT (OLD.telefono <=> NEW.telefono) THEN
  INSERT INTO auditoria_paciente (id_paciente, campo, valor_anterior, valor_nuevo,
                                  usuario_sesion, usuario_privilegios)
  VALUES (NEW.id_paciente, 'telefono', OLD.telefono, NEW.telefono, USER(),
CURRENT_USER());
END IF;
```

Figura 13

Registro de auditoría tras modificar el teléfono de un paciente



Nota. Captura propia en MySQL Workbench.

6. Vistas

`vw_citas_detalle` reúne cita, paciente, médico, especialidad y centro de salud con INNER JOIN sobre cinco tablas, y evita que quien solo consulta tenga que leer las tablas base. `vw_pacientes_publico` enmascara el documento, el teléfono y el correo con `fn_enmascarar` y agrega la categoría de edad con `fn_clasificar_edad`. La comparación con los datos reales de la tabla está en el Anexo C.

```
CREATE OR REPLACE SQL SECURITY DEFINER VIEW vw_pacientes_publico AS
SELECT id_paciente, nombre, apellido,
       fn_enmascarar(documento, 4) AS documento,
       fn_enmascarar(telefono, 4)  AS telefono,
       CONCAT(LEFT(correo, 2), '***', SUBSTRING(correo, LOCATE('@', correo))) AS
correo,
       fn_clasificar_edad(TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE())) AS
categoria_edad
  FROM paciente;
```

Figura 14

Vista vw_citas_detalle (JOIN de cinco tablas)

```
1 •  SELECT * FROM vw_citas_detalle LIMIT 10;
```

Show the Find panel for the editor

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id_cita	fecha	hora	estado	paciente	medico	especialidad	centro_salud
1	2026-08-03	08:00:00	Atendida	Juan Pérez	Andrés Gómez	Medicina General	Centro de Salud San Jerónimo
11	2026-09-17	15:00:00	Cancelada	Juan Pérez	Carlos Ramírez	Cardiología	Centro de Salud Santa Ana
2	2026-08-05	09:00:00	Atendida	Sofía Ospina	Laura Martínez	Pediatría	Centro de Salud El Prado
14	2026-09-30	10:30:00	Programada	Sofía Ospina	Laura Martínez	Pediatría	Centro de Salud El Prado
3	2026-08-10	14:00:00	Atendida	Luis Cardona	Carlos Ramírez	Cardiología	Centro de Salud Santa Ana
12	2026-09-18	10:00:00	No asistio	Luis Cardona	Andrés Gómez	Medicina General	Centro de Salud San Jerónimo
4	2026-08-12	08:30:00	Atendida	María Giraldo	Diana López	Ginecología	Centro de Salud Zaragoza
5	2026-08-18	10:00:00	Atendida	Jorge Valencia	Felipe Torres	Odontología	Centro de Salud Guadalupe
13	2026-09-28	09:00:00	Programada	Jorge Valencia	Andrés Gómez	Medicina General	Centro de Salud San Jerónimo
6	2026-08-24	15:00:00	Atendida	Natalia Restrepo	Valentina Rojas	Dermatología	Centro de Salud Bellavista

Nota. Captura propia en MySQL Workbench.

Figura 15

Vista vw_pacientes_publico con datos personales enmascarados

```
1   SELECT * FROM vw_pacientes_publico LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id_paciente	nombre	apellido	documento	telefono	correo	categoria_edad
1	Juan	Pérez	******5601	******0000	ju***@mail.com	Adulto
2	Sofía	Ospina	******5602	******0002	ac***@mail.com	Niño
3	Luis	Cardona	******5603	******0003	lu***@mail.com	Adulto mayor
4	María	Giraldo	******5604	******0004	ma***@mail.com	Adulto
5	Jorge	Valencia	******5605	******0005	jo***@mail.com	Adulto
6	Natalia	Restrepo	******5606	******0006	na***@mail.com	Adulto
7	Daniel	Quintero	******5607	******0007	da***@mail.com	Adulto
8	Carolina	Muñoz	******5608	******0008	ca***@mail.com	Adulto
9	Hernando	Salazar	******5609	******0009	NULL	Adulto mayor
10	Isabela	Arango	******5610	******0010	is***@mail.com	Adulto

Nota. Captura propia en MySQL Workbench.

7. Seguridad

7.1 Matriz de seguridad y usuarios

Se definieron tres roles con el principio de mínimo privilegio. El rol de consulta solo lee las vistas; el de recepción hereda esa consulta y solo puede ejecutar procedimientos (no tiene INSERT sobre las tablas); el administrador gestiona los datos tabla por tabla y, para que la auditoría no pueda alterarse, solo tiene SELECT sobre `auditoria_paciente`.

Tabla 5

Matriz de seguridad (rol × objeto × privilegio)

Objeto	rol_consulta_citas	rol_recepcion_citas	rol_admin_citas
`vw_pacientes_publico`, `vw_citas_detalle`	SELECT	SELECT (heredado)	SELECT
`paciente`, `cita`, `historial_medico`, `medico`, `especialidad`, `centro_salud`, `recordatorio`	-	-	SELECT, INSERT, UPDATE, DELETE
`auditoria_paciente`	-	-	SELECT (solo lectura)
`sp_agendar_cita_segura`, `sp_cancelar_cita`, `fn_validar_login_citas`	-	EXECUTE	EXECUTE
Demás procedimientos y funciones	-	-	EXECUTE
DROP TABLE, CREATE USER	-	-	-

Tabla 6

Usuarios y política de cuenta

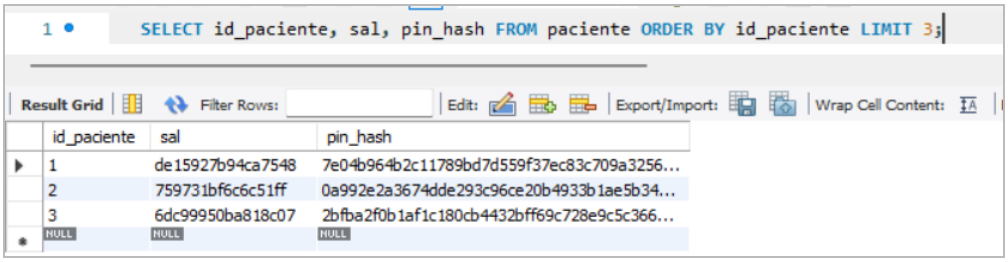
Usuario	Rol por defecto	Política de cuenta
`lector_citas`	`rol_consulta_citas`	La contraseña vence cada 90 días
`recepcion_citas`	`rol_recepcion_citas`	Máximo 2 conexiones simultáneas y 500 consultas por hora
`gestor_citas`	`rol_admin_citas`	Sin restricciones adicionales

7.2 Hash con sal y cifrado

El PIN de acceso del paciente no se guarda en claro: se almacena `SHA2(CONCAT(sal, pin), 256)` con una sal aleatoria distinta por paciente, de modo que dos pacientes con el mismo PIN tienen hashes diferentes (Figura siguiente). La función `fn_validar_login_citas` compara hashes y nunca el PIN. Las observaciones médicas se cifran con AES (`AES_ENCRYPT`) en `observaciones_cifrado` y solo se leen con la llave correcta mediante `AES_DECRYPT`.

Figura 16

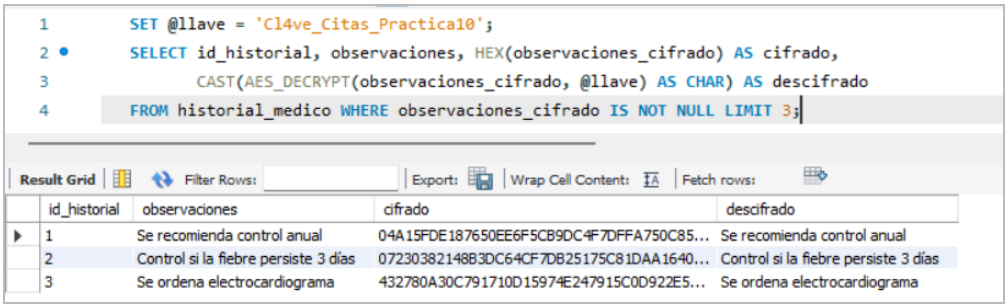
Sal y hash del PIN por paciente



Nota. Captura propia en MySQL Workbench.

Figura 17

Observaciones médicas en claro, cifradas (HEX) y descifradas con la llave



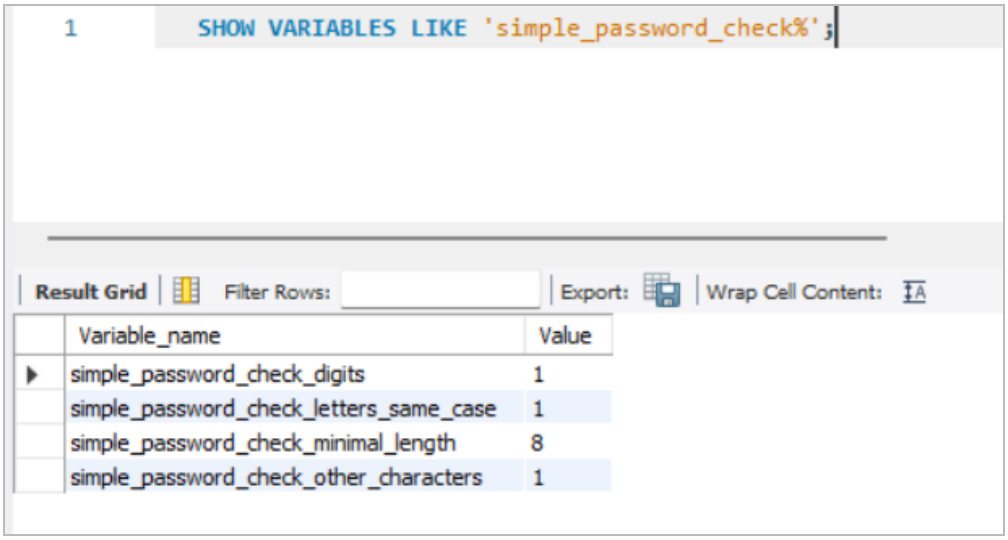
Nota. Captura propia en MySQL Workbench.

7.3 Política de contraseñas

Se activó el plugin `simple_password_check`, que exige mínimo 8 caracteres, un dígito, mayúsculas y minúsculas y un carácter especial. Una contraseña débil es rechazada con el error 1819.

Figura 18

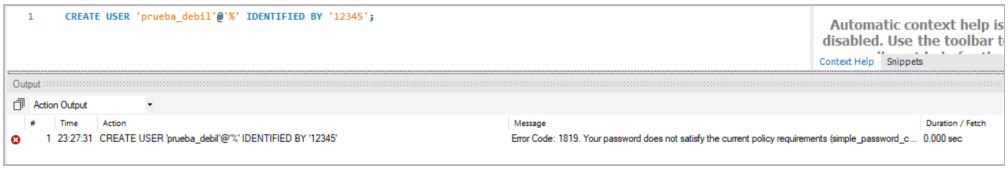
Variables de la política de contraseñas



Nota. Captura propia en MySQL Workbench.

Figura 19

Creación de un usuario con contraseña débil rechazada



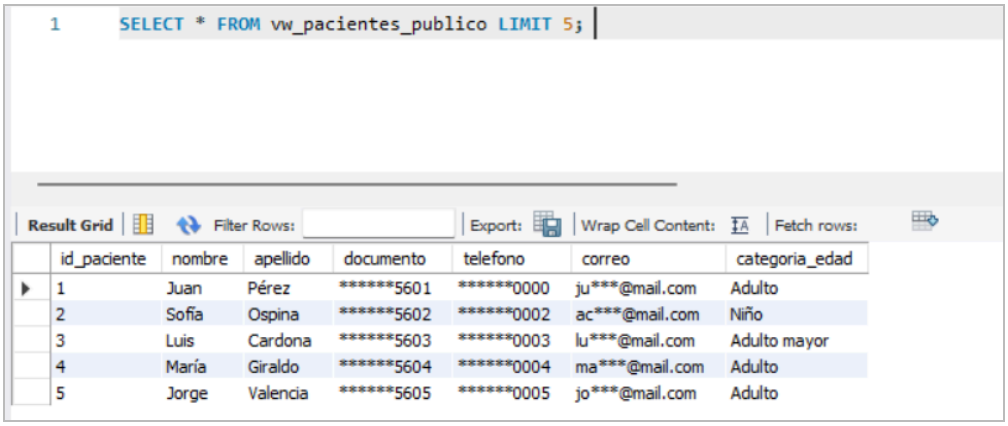
Nota. Captura propia en MySQL Workbench.

7.4 Pruebas con cada usuario

Cada usuario se conectó desde su propia sesión en Workbench. La identidad de cada sesión (`CURRENT_USER()` y `CURRENT_ROLE()`) se muestra en el Anexo A.

Figura 20

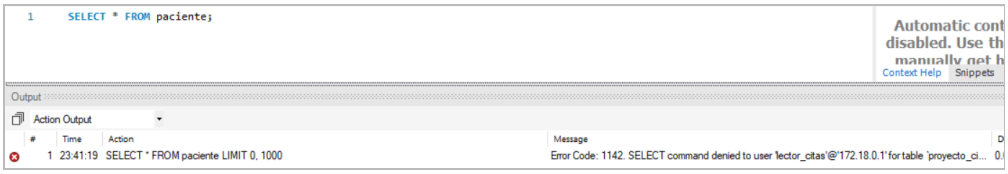
lector_citas: consulta permitida a la vista enmascarada



Nota. Captura propia en MySQL Workbench.

Figura 21

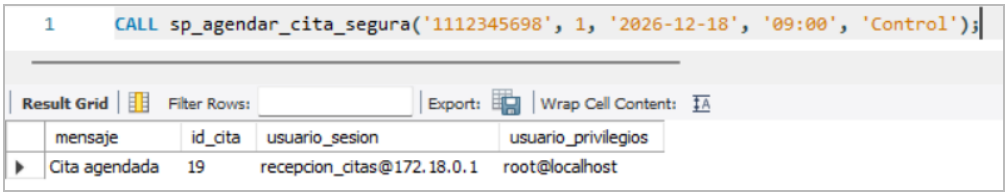
lector_citas: acceso denegado a la tabla paciente (error 1142)



Nota. Captura propia en MySQL Workbench.

Figura 22

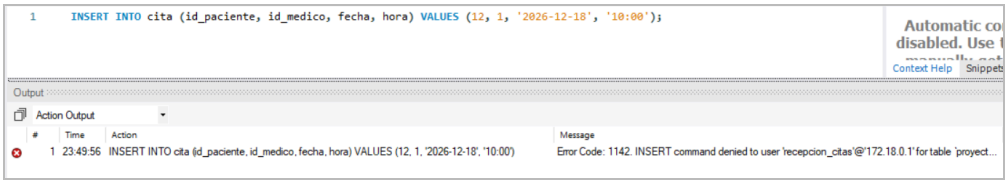
recepcion_citas: agendamiento permitido con sp_agendar_cita_segura (SQL SECURITY DEFINER)



Nota. Captura propia en MySQL Workbench. `usuario_sesion` es recepcion_citas y `usuario_privilegios` es root, porque el procedimiento se ejecuta con los privilegios de su creador.

Figura 23

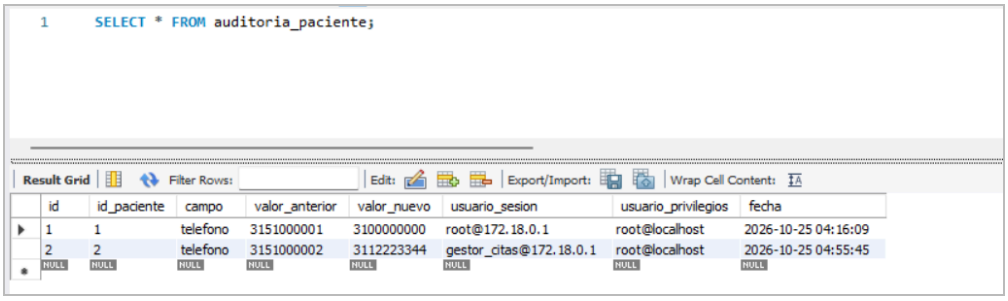
recepcion_citas: INSERT directo sobre cita denegado (error 1142)



Nota. Captura propia en MySQL Workbench.

Figura 24

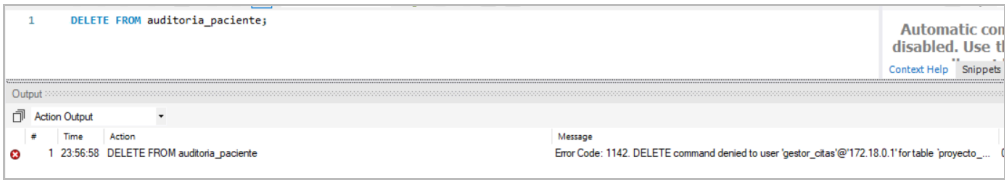
gestor_citas: UPDATE permitido, registrado en la auditoría con su usuario



Nota. Captura propia en MySQL Workbench. La fila 2 corresponde al UPDATE hecho por gestor_citas.

Figura 25

gestor_citas: DELETE sobre la auditoría denegado (error 1142)



Nota. Captura propia en MySQL Workbench.

8. Dificultades y Próximos Pasos

Dificultades y cómo se resolvieron:

- **Scripts que no corrían desde cero.** El script de seguridad usaba funciones guardadas fuera del repositorio. Se movieron a `sql/funciones.sql` y se creó `cargar_bd.sh`; se verificó con `docker compose down -v` y una carga completa sin errores (8 tablas, 2 vistas, 6 procedimientos, 5 funciones y 3 triggers).
- **Identificador de la cita.** `sp_agendar_cita` usaba `MAX(id_cita)`, que con conexiones simultáneas puede devolver el id de otra sesión; se cambió por `LAST_INSERT_ID()`.
- **Plugin de contraseñas.** En una base nueva `simple_password_check` no estaba instalado y la creación de usuarios fallaba; ahora `cargar_bd.sh` lo instala.
- **Conexiones de prueba.** Una conexión sin esquema por defecto devolvió el error 1046 en lugar de la prueba de permisos; se corrigió ejecutando `USE proyecto_citas`.
- **Permisos tras recrear procedimientos.** Al eliminar un procedimiento se pierden los privilegios EXECUTE concedidos sobre él; por eso el script de seguridad se ejecuta siempre al final.

Pendiente para el proyecto final:

- Estados de la cita como tabla independiente (`estado_cita`), para agregar o eliminar estados sin modificar `cita`, según lo sugerido por el docente.

- Franjas horarias por médico (`franja_horaria`) en lugar de un único horario de inicio y fin, también sugerido por el docente.
- Sacar contraseñas y llave de cifrado de los scripts hacia variables de entorno o un gestor de secretos, y restringir las cuentas a hosts específicos en lugar de `%`.
- Habilitar TLS, crear índices según las consultas más frecuentes e implementar el registro de la atención (historial y estado Atendida) en una sola operación.

9. Uso de Inteligencia Artificial

Se utilizó Claude (Anthropic) como apoyo para revisar los lineamientos del avance frente al estado del proyecto, redactar y corregir scripts SQL (funciones, validaciones con `SIGNAL`, `cargar_bd.sh`), y estructurar este documento. Los resultados se verificaron ejecutando todos los scripts en Docker sobre una base limpia, probando cada caso de éxito y de error en MySQL Workbench y con cada usuario, y revisando manualmente el contenido; todas las capturas son del entorno propio. Las decisiones de diseño, como no modificar el esquema en este corte, las tomó la autora.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Cano Sánchez, J. J. (2026). *Avance 2 del proyecto de semestre – Segundo corte* [Material de clase]. Bases de Datos II, Corporación de Estudios Tecnológicos del Norte del Valle.

MariaDB Foundation. (s. f.-a). *AES_ENCRYPT*. MariaDB Knowledge Base. https://mariadb.com/kb/en/aes_encrypt/

MariaDB Foundation. (s. f.-b). *CREATE FUNCTION*. MariaDB Knowledge Base. https://mariadb.com/kb/en/create-function/

MariaDB Foundation. (s. f.-c). *CREATE PROCEDURE*. MariaDB Knowledge Base. https://mariadb.com/kb/en/create-procedure/

MariaDB Foundation. (s. f.-d). *CREATE TRIGGER*. MariaDB Knowledge Base. https://mariadb.com/kb/en/create-trigger/

MariaDB Foundation. (s. f.-e). *CREATE VIEW*. MariaDB Knowledge Base. https://mariadb.com/kb/en/create-view/

MariaDB Foundation. (s. f.-f). *Roles overview*. MariaDB Knowledge Base. https://mariadb.com/kb/en/roles_overview/

MariaDB Foundation. (s. f.-g). *SHA2*. MariaDB Knowledge Base. https://mariadb.com/kb/en/sha2/

MariaDB Foundation. (s. f.-h). *SIGNAL*. MariaDB Knowledge Base. https://mariadb.com/kb/en/signal/

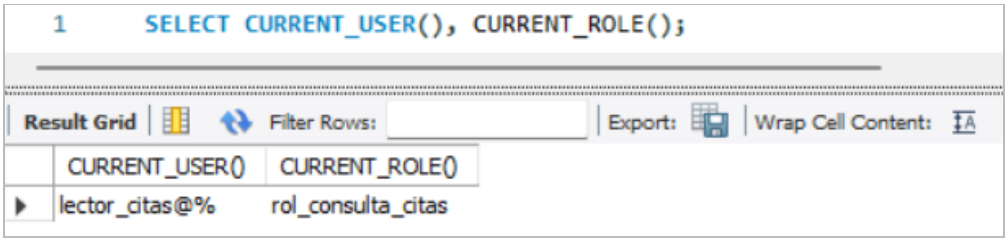
MariaDB Foundation. (s. f.-i). *Simple Password Check Plugin*. MariaDB Knowledge Base. https://mariadb.com/kb/en/simple-password-check-plugin/

Oracle Corporation. (s. f.). *MySQL Workbench manual*. https://dev.mysql.com/doc/workbench/en/

Anexo A. Identidad de cada sesión de prueba

Figura 26

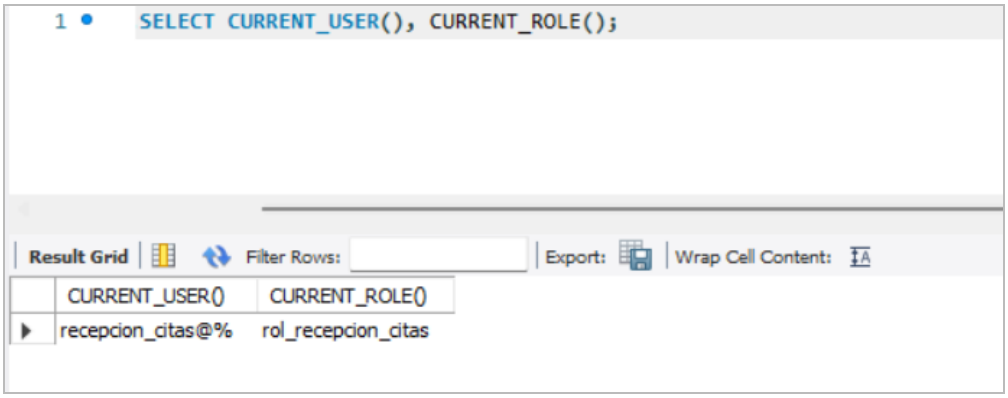
Sesión de lector_citas con su rol activo



Nota. Captura propia en MySQL Workbench.

Figura 27

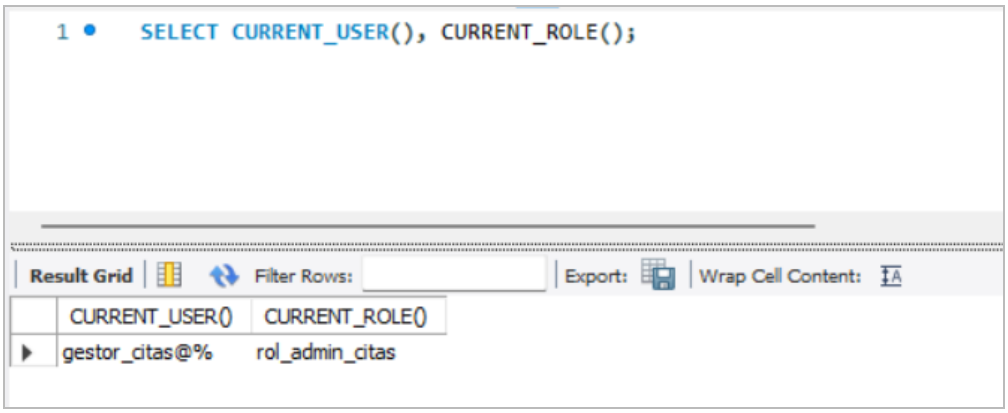
Sesión de recepcion_citas con su rol activo



Nota. Captura propia en MySQL Workbench.

Figura 28

Sesión de gestor_citas con su rol activo

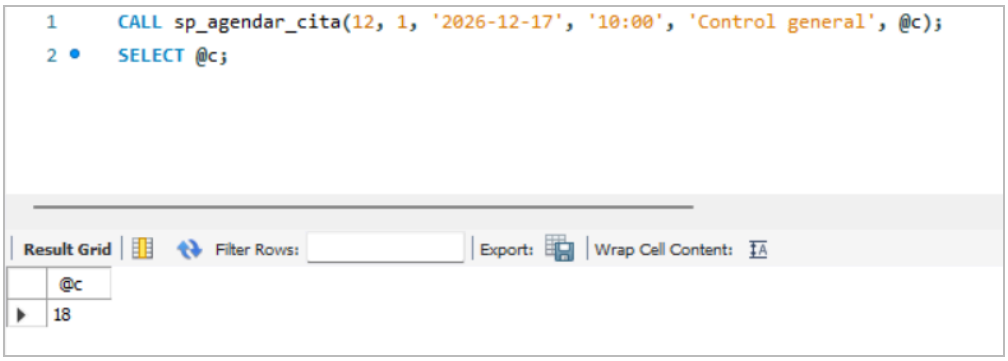


Nota. Captura propia en MySQL Workbench.

Anexo B. Evidencias adicionales de triggers

Figura 29

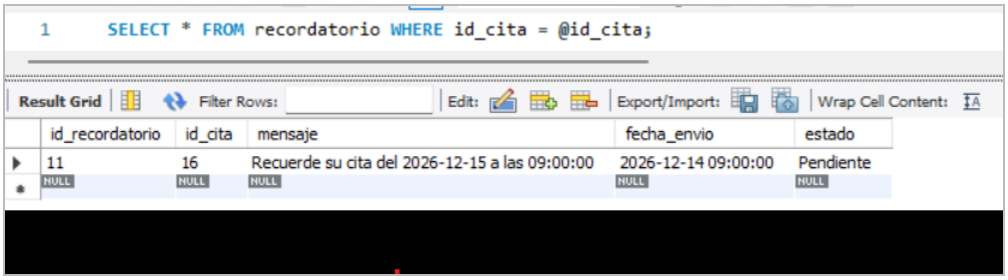
Cita válida agendada correctamente (id 18)



Nota. Captura propia en MySQL Workbench.

Figura 30

Recordatorio generado automáticamente para la cita 16

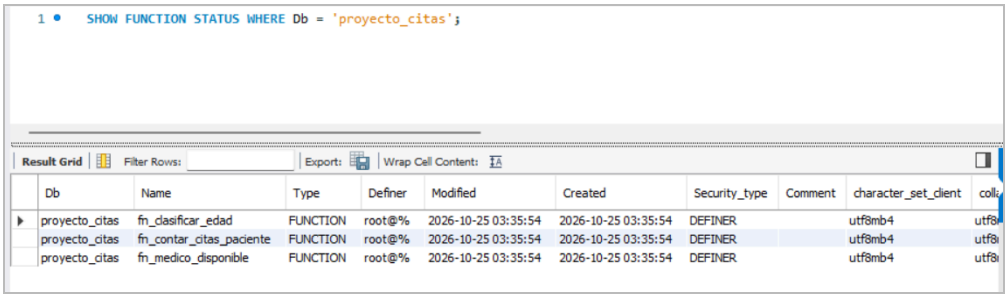


Nota. Captura propia en MySQL Workbench.

Anexo C. Funciones creadas y comparación de la vista

Figura 31

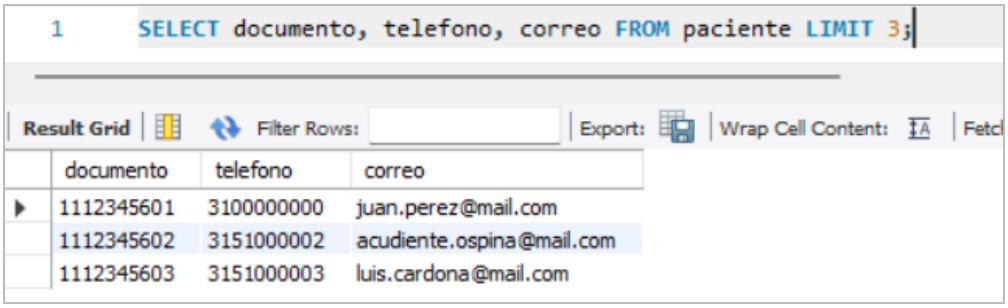
Funciones almacenadas creadas en la base proyecto_citas



Nota. Captura propia en MySQL Workbench.

Figura 32

Datos reales de la tabla paciente, para comparar con la vista enmascarada



Nota. Captura propia en MySQL Workbench.